DAY 5: SQL CODING CHALLENGE –JOINS & BUILDIN FUNTION

Database: School database

```
-- 1CREATE DATABASE

CREATE DATABASE StudentsDB;
USE StudentsDB;

-- CREATE TABLES

CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Age INT,
    City VARCHAR(50)
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    Duration VARCHAR(20)
);

CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
```

```
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);



-- INSERT SAMPLE DATA

INSERT INTO Students VALUES
(1, 'Aarav', 21, 'Chennai'),
(2, 'Meera', 22, 'Bangalore'),
(3, 'Karthik', 23, 'Hyderabad'),
(4, 'Divya', 21, 'Delhi');

INSERT INTO Courses VALUES
(101, 'Data Analytics', '3 Months'),
(102, 'Python Programming', '2 Months'),
(103, 'SQL Basics', '1 Month');

INSERT INTO Enrollments VALUES
(1001, 1, 101, '2025-05-10'),
(1002, 2, 102, '2025-06-01'),
(1003, 3, 103, '2025-06-15');
```

Question 1 – INNER JOIN

Scenario:
Show students with their enrolled course names.

Task:
Write a query joining **Students** and **Courses** through **Enrollments**.

Expected Output:
Only students who have valid enrollments (common in both tables).

Question 2 – LEFT JOIN and RIGHT JOIN

Scenario:
List all students and their courses, including those without matches.

Task:
Use both **LEFT JOIN** and **RIGHT JOIN** between **Students** and **Enrollments**.

Expected Output:
All students/courses are shown, with **NULL** where no match exists.

Question 3 – ROUND()

Scenario:
While preparing numeric reports, analysts need to round off decimal values.

Task:
Round the value **123.4567** to **two decimal places** using the **ROUND()** function.

Expected Output:
Display a single column showing **123.46**.

Question 4 – ABS() & MOD()

Scenario:
The HR team wants to calculate absolute values and remainders for data validation.

Task:
Use **ABS()** to convert negative numbers to positive and **MOD()** to find the remainder when **25 is divided by 4**.

Expected Output:
Display absolute value and remainder results.

Question 5 – CONCAT()

Scenario:
The placement cell wants a full description combining each student's name and city.

Task:
Use **CONCAT()** to merge **StudentName** and **City** into one column like "**Aarav from Chennai**."

Expected Output:
Full_Description showing combined values.

Question 6 – LENGTH()

Scenario:
The admin wants to find names with length greater than a certain value for formatting.

Task:
Use **LENGTH()** to display each student's name along with its character count.

Expected Output:
`StudentName | Name_Length`

Question 7 – REPLACE()

Scenario:
Course titles need updating — every occurrence of "SQL" should be replaced with "Database."

Task:
Use **REPLACE()** to modify course names in the **Courses** table.

Expected Output:
`CourseName | Updated_CourseName`

Question 8 – SUBSTRING()

Scenario:
For generating student codes, you need the first 3 letters of each name.

Task:
Use **SUBSTRING()** to extract the first 3 characters from **StudentName**.

Expected Output:
`StudentName | Code_Prefix`

Question 9 – UPPER() & LOWER()

Scenario:
Standardize name formatting for email IDs and certificates.

Task:
Use **UPPER()** and **LOWER()** to show each name in uppercase and lowercase.

Expected Output:
`StudentName | UPPER_Name | LOWER_Name`

Question 10 – DATE FUNCTIONS (NOW, DATEDIFF, DATE_ADD)

Scenario:
The enrollment team needs to calculate report time, duration, and follow-up dates.

Task:

1. Use **NOW()** to display the current date and time.
2. Use **DATEDIFF()** to find the number of days between `'2025-06-01'` and `'2025-05-10'`.
3. Use **DATE_ADD()** to add 10 days to each student's **EnrollmentDate**.

Expected Output:
`StudentName | EnrollmentDate | FollowUp_Date | Days_Difference | Current_DateTime`